

User Registration and Login System using JWT (JSON Web Token) in Python (Flask framework)

Step 1: Install Dependencies

- Open your terminal and install these packages:

```
pip install Flask flask-bcrypt Flask-JWT-Extended
```

Step 2: Project Structure

```
jwt_auth_app/ |
```

```
|— app.py
```

```
|— users.db (auto-created later)
```

- **Step 3: Import Required Libraries**

```
from flask import Flask, request, jsonify
from flask_bcrypt import Bcrypt
from flask_jwt_extended import (
    JWTManager, create_access_token,
    jwt_required, get_jwt_identity)
import sqlite3
```

- **Step 4: Initialize Flask App and JWT**

```
app = Flask(__name__)
```

```
app.config['JWT_SECRET_KEY'] =  
    'your_secret_key_here'
```

```
# change this to a strong key
```

```
bcrypt = Bcrypt(app)
```

```
jwt = JWTManager(app)
```

Step 5: Database Setup (SQLite for simplicity)

```
def init_db():  
    conn = sqlite3.connect('users.db')  
    c = conn.cursor()  
    c.execute("""  
        CREATE TABLE IF NOT EXISTS users (  
            id INTEGER PRIMARY KEY AUTOINCREMENT,  
            username TEXT UNIQUE NOT NULL,  
            password TEXT NOT NULL  
        )  
    """)  
    conn.commit()  
    conn.close()  
    init_db()
```

Step 6: User Registration Endpoint

```
@app.route('/register', methods=['POST'])
def register():
    data = request.get_json()
    username = data.get('username')
    password = data.get('password')
    if not username or not password:
        return jsonify({"error": "Username and password are required"}), 400
    hashed_pw = bcrypt.generate_password_hash(password).decode('utf-8')
    try:
        conn = sqlite3.connect('users.db')
        c = conn.cursor()
        c.execute('INSERT INTO users (username, password) VALUES (?, ?)', (username,
            hashed_pw))
    conn.commit()
    conn.close()
    return jsonify({"message": "User registered successfully"}), 201
except sqlite3.IntegrityError:
    return jsonify({"error": "Username already exists"}), 409
```

Step 7: Login Endpoint (Generate JWT Token)

```
@app.route('/login', methods=['POST'])
def login():
    data = request.get_json()
    username = data.get('username')
    password = data.get('password')
    conn = sqlite3.connect('users.db')
    c = conn.cursor()
    c.execute('SELECT password FROM users WHERE username=?', (username,))
    user = c.fetchone()
    conn.close()
    if user and bcrypt.check_password_hash(user[0], password):
        token = create_access_token(identity=username)
        return jsonify({"token": token}), 200
    else:
        return jsonify({"error": "Invalid credentials"}), 401
```

Step 8: Protected Route (JWT Required)

```
@app.route('/profile', methods=['GET'])
@jwt_required()def profile():
    current_user = get_jwt_identity()
    return jsonify({"logged_in_as": current_user}),
    200
```

- Step 9: Run the App

```
if __name__ == '__main__':  
    app.run(debug=True)
```

- **Step 10: Test the API using Postman or Curl**

- **1. Register:**

POST /register

Content-Type: application/json

{

"username": "saranya",

"password": "mypassword123"

}

- 2. Login:

POST /login

Content-Type: application/json

{

 "username": "saranya",

 "password": "mypassword123"

}

- **Returns:**

```
{  
  "token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1b290eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1b290eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.  
}
```

3. Access protected route:

GET /profile

Authorization: Bearer <token>

Returns:

```
{  
  "logged_in_as": "saranya"  
}
```

JWT

Types of JWT

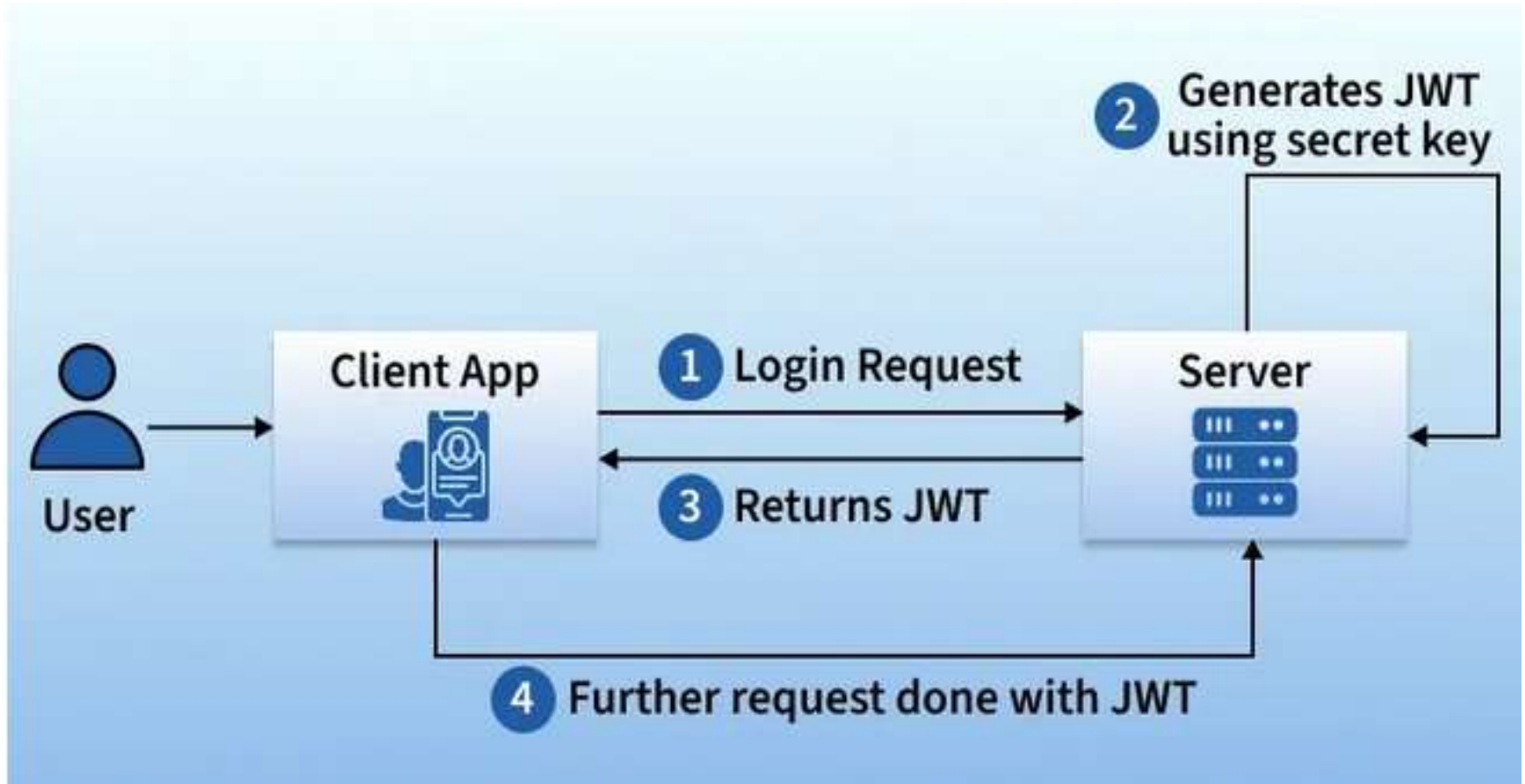
There are **two main types** of JWT based on how the payload is protected- JWS and JWE. Let's learn more about them.

JWS (JSON Web Signature)

- JWS is a type of JWT where the payload (data) is digitally signed, ensuring integrity and authenticity of the token.
- The payload is Base64URL encoded and signed using a secret or private key.
- It is not encrypted, meaning the contents can be read by anyone who has the token.
- Commonly used in authentication and authorization scenarios like OAuth 2.0 access tokens.

Use Case: Verifying that the data has not been tampered with.

How JWT token Works?



- **JWE (JSON Web Encryption)?**
- JWE is a type of JWT where the payload is encrypted, ensuring confidentiality in addition to integrity.
- The entire payload is encrypted using a public key or shared secret.
- Only the intended recipient can decrypt and read the token contents.
- Less common than JWS, but ideal for sensitive data transmission.

Use Case: Protecting personal or confidential information during transit.

• JWT vs JWS vs JWE – Comparison Table

Feature	JWT (General)	JWS (Signed JWT)	JWE (Encrypted JWT)
Security Focus	Token Format	Integrity, authenticity	Confidentiality + integrity
Payload	Not specified	Base64URL encoded (readable)	Encrypted (not readable)
Signature	Optional	Required	Encrypted along with payload
Encryption	Optional	Not encrypted	Fully encrypted
Use Case	ID and Access Tokens	OAuth 2.0, OpenID Connect	Highly sensitive information

Structure of a JWT

- A JWT consists of **three parts**, separated by dots (.):

xxxxx.yyyyyy.zzzzzz

1. Header — contains metadata about the token

Example:

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

- alg → signing algorithm (e.g., HS256)
- typ → token type (JWT)

2. **Payload** — contains claims (information) about the user

Example:

```
{  
  "sub": "1234567890",  
  "email": "user@example.com",  
  "iat": 1718888888,  
  "exp": 1718892488  
}
```

- sub → subject (usually user ID)
- iat → issued at
- exp → expiration time

3. Signature — used to verify that the token hasn't been changed

Example process:

HMACSHA256(

base64UrlEncode(header) + "." +
base64UrlEncode(payload),

secret

)

- **JWT Signing Algorithms Explained**
- JWTs rely on signing algorithms to ensure tokens are valid and untampered. These algorithms play a critical role in verifying identity, especially in authentication and authorization workflows.
- There are **two core types** of signing mechanisms used in JWTs: symmetric and asymmetric algorithms.

JWT Algorithm Types: Symmetric vs. Asymmetric

- Understanding JWT algorithm types helps you choose the right method for your security needs.
- **1. Symmetric Algorithms (HMAC - HS256)**
- Symmetric algorithms like **HS256** use the same secret key for both signing and verifying a token.
- **Easy to implement**
- **Risky in distributed systems** because the key must be shared between parties

```
const jwt = require('jsonwebtoken');
```

```
const token = jwt.sign({ userId: '12345' }, 'your-256-bit-secret', { algorithm: 'HS256' });
```

2. Asymmetric Algorithms (RSA, ECDSA)

- Asymmetric algorithms such as **RS256**, **ES256**, and **PS256** use a private–public key pair.
- **More secure** for public APIs and microservices
- The **private key signs** the token, while the **public key verifies** it and can be safely shared

```
const fs = require('fs');
```

```
const privateKey =  
  fs.readFileSync('./private.key');
```

```
const token = jwt.sign({  userId:  '12345'  },  
  privateKey, { algorithm: 'RS256' });
```

- **JWT Algorithm List and Use Cases**
- Here's a **JWT algorithm list** that highlights key algorithms used in production, with a quick comparison to help you decide:

Algorithm	Type	Key Used	Strengths
HS256	Symmetric	Shared secret	Fast, simple to implement
RS256	Asymmetric	RSA key pair	Secure for third-party verification
ES256	Asymmetric	EC key pair	Lightweight, efficient
PS256	Asymmetric	RSA-PSS	High-level cryptographic strength

- **How to Choose the Right JWT Algorithm**
- Use **HS256** for small, internal apps where all services can securely share a secret key.
- Use **RS256** or **ES256** for apps with **external integrations, public APIs, or microservices**, where stronger verification and key separation are needed.
- Choose **PS256** when compliance and modern cryptographic standards (like FIPS or GDPR) are essential.

- **Implementation of JWT using LoginRadius APIs**
- To implement JWT with LoginRadius:
- **Step 1: Configure a JWT App**
- Set up a JWT app in your [LoginRadius Admin Console](#). Follow the JWT Admin Console Configuration guide.

Applications

Authentication

Customers

Security

Single Protection

Password Policy

Security Questions

Multi Factor Authentication

Session Management

Role-based Authentication

Branding

Integration

Orchestration

—

14 Customers

Session Management

Configure & Manage default settings for user sessions [Documentation](#)

Access Token

Configure the access token lifetime as per your use cases upon token expiry, the customer must login again. It can be set from 1 to 12000 minutes (20 days). Contact support to set a longer token expiry.

Access Token Lifetime (Minutes) ⓘ

Save

Reset

Refresh Token

Configure the refresh token lifetime to obtain additional access tokens as per your use cases. The default lifetime is 30 days (2000 minutes) and can be extended to 300 days (120000 minutes). Contact support to set a longer token expiry.

Refresh Token Lifetime (Minutes)

Save

Reset

Step 2: Use APIs to Retrieve JWT

- If you are directly implementing your Login forms or already have an access token or want to generate a JWT based on email/username/Phone number or a password, you can leverage the following APIs:
- [JWT Token](#): This GET API is used to exchange access tokens with your JWT.
- [JWT Token by Email](#): This API is used to get a JWT by Email and Password.
- [JWT Token by Username](#): This API is used to get JWT by Username and password.
- [JWT Token by Phone](#): This API is used to get JWT by Phone and password.
- **API Response Example:**
 - {
 - "signature": "<JWTresponse>"
 - }
- These tokens can then be used in your client app for authenticated requests.

Best Practices for Secure JWT Authentication

- To implement JWT securely, follow these key practices:
- **Keep Signing Keys Secure**
- Private keys or secrets used to sign JWTs must be stored securely.
- **Avoid Sensitive Data in Payload**
- Payload is only base64 encoded—not encrypted. Do not include passwords, PII, or credentials unless using encrypted JWT (JWE).
- **Limit Token Claims**
- Include only essential claims in the token to reduce size and exposure.
- **Use HTTPS**
- Always transmit JWTs over HTTPS to prevent man-in-the-middle attacks.
- **Set Short Expiry Times**
- Use short exp durations and implement refresh tokens to reduce impact if a token is compromised.
- **Implement Token Revocation**
- Use jti with a blacklist or maintain a revocation strategy for enhanced control.

Why Are JWTs Important for Authentication and Security?

- JWTs offer numerous benefits in authentication systems:
- **Stateless Authentication** – No need to maintain session state on the server.
- **Scalability** – Suitable for microservices and distributed systems.
- **Tamper Resistance** – Digitally signed tokens ensure data integrity.
- **Performance** – Reduces server load and database dependencies.
- **Cross-Platform Support** – Easily used across web, mobile, and API ecosystems.
- **Enhanced Security** – Signed tokens ensure authenticity and tamper-proof data.
- **Developer Convenience** – Simplifies session management.
- JWTs are widely adopted in OAuth 2.0, OpenID Connect, and API security implementations.